

{dev/talles}

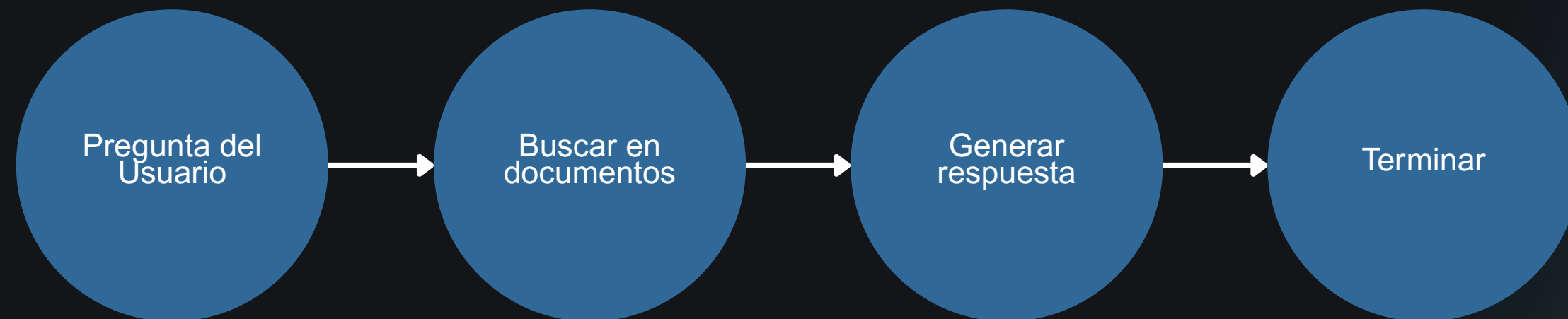
LangGraph

Por: Ricardo Cuellar



¿Qué hemos hecho hasta ahora?

Cuando construimos sistemas de IA con LCEL los flujos fueron lineales. El sistema siempre hace los mismos pasos en el mismo orden, sin importar el contexto.

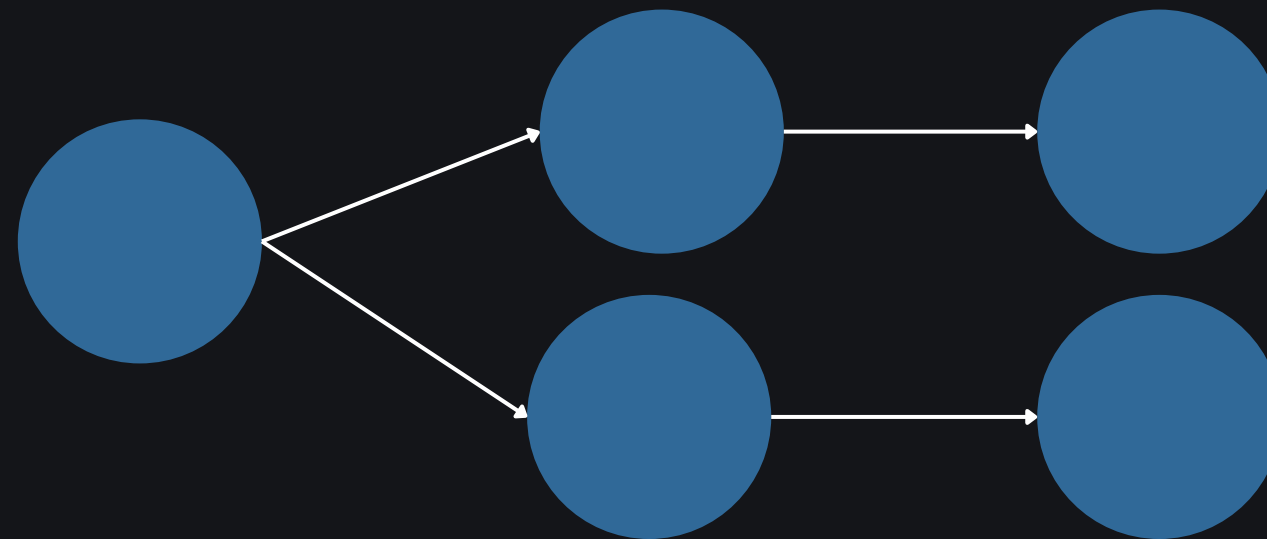




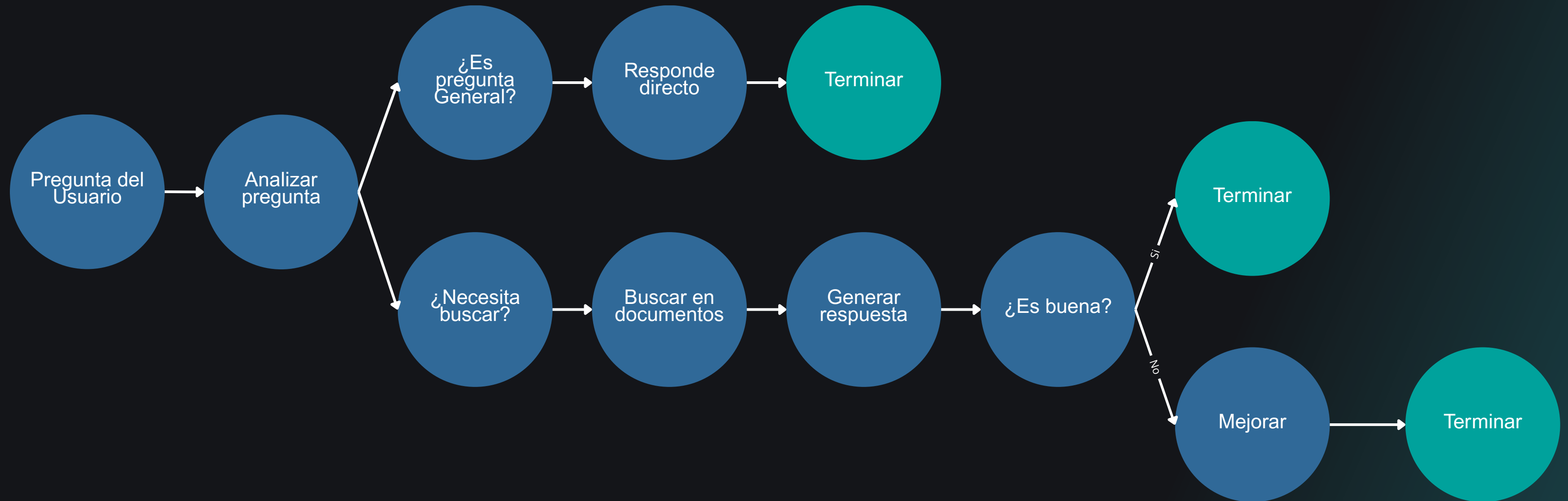
¿Qué es el LangGraph?

Es una librería de Python que permite construir sistemas de IA con flujos cíclicos, decisiones condicionales y estado compartido.

Creado por el equipo de LangChain y lanzado en 2024, ahora es el estándar de la industria para construir agentes y sistemas complejos de IA.



LangGraph





Conceptos fundamentales

- State (Estado)
- Node (Nodo)
- Edge (Arista)
- StateGraph y el compilador



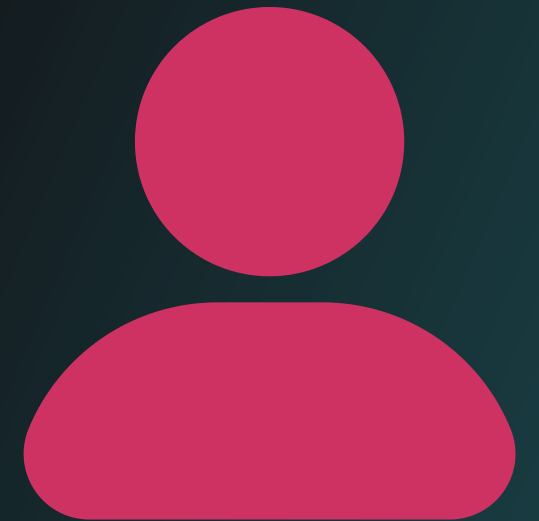
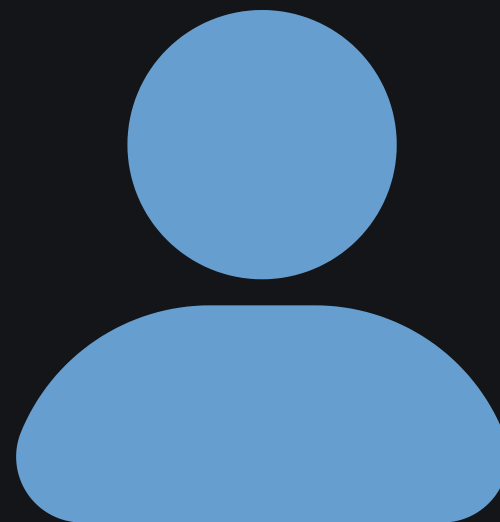
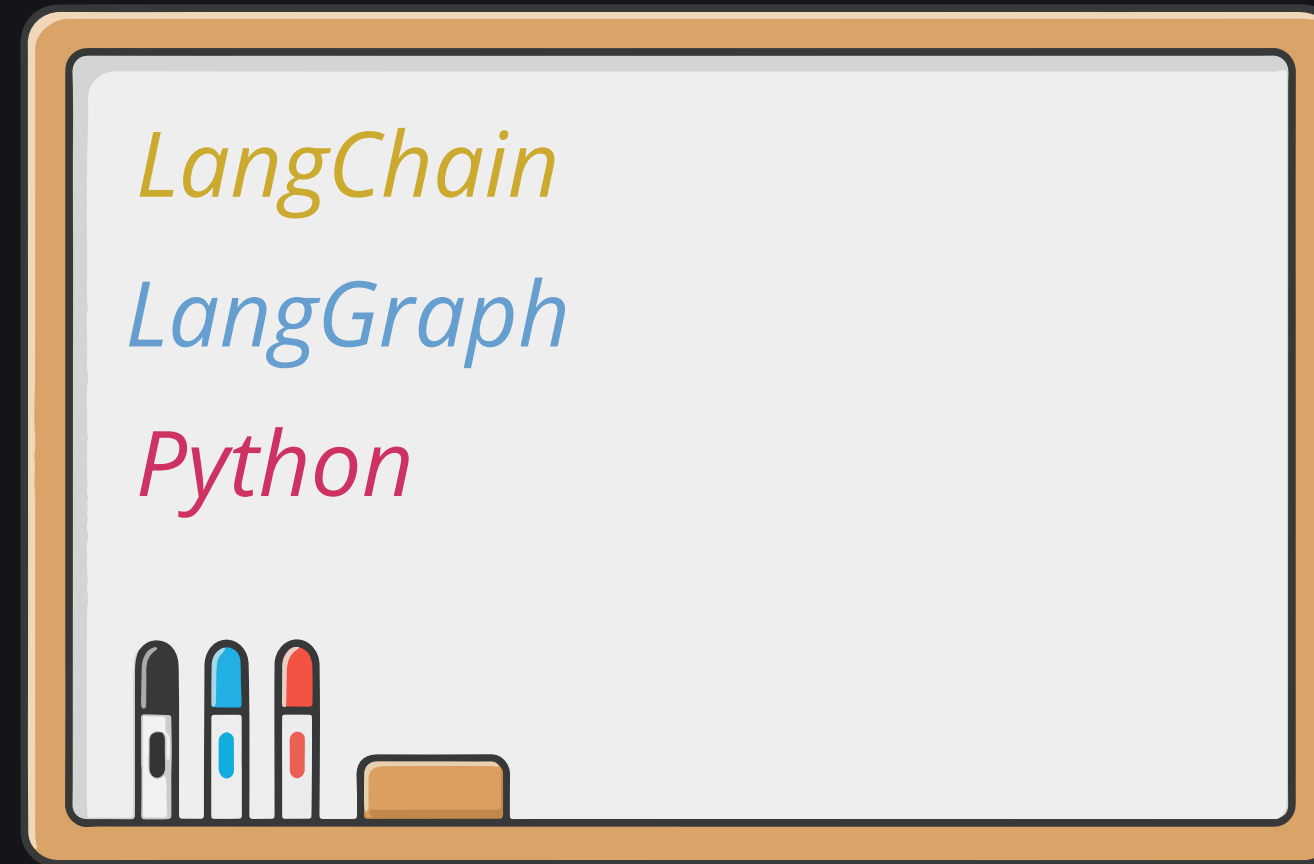
State (Estado)

Es un diccionario tipado que viaja por todo el sistema. Cada nodo puede leer cualquier campo del estado y escribir solo los campos que le corresponden.

```
state.py

class RAGAgentState(TypedDict):
    messages: list      # Historial de la conversación
    question: str       # La pregunta actual
    retrieved_docs: list # Documentos encontrados
    response: str       # La respuesta generada
    needs_retrieval: bool # ¿Necesita buscar en documentos?
    sources: list       # Fuentes usadas
```

State (Estado) - Analogía



Node (Nodo)

Un nodo es simplemente una función de Python que recibe el estado completo y devuelve un diccionario con solo los campos que modifica. Es la unidad de trabajo del sistema.

```
node.py

def node_retrieve(state: RAGAgentState) -> dict:
    # Lee del estado
    question = state["question"]

    # Hace su trabajo específico
    docs = buscar_en_chromadb(question)

    # Devuelve SOLO lo que cambió
    return {
        "retrieved_docs": docs,
        "sources": extraer_metadatos(docs)
    }
```



Edge (Arista)

Una arista es la conexión entre nodos. Existen dos tipo:

- Edge normal
- Edge condicional



Edge - Normal

Siempre va del nodo A al nodo B, sin condiciones.



edge.py

```
builder.add_edge("retrieve", "generate")
```

```
# Después de retrieve, SIEMPRE va a generate
```

Edge - Condicional

Una función decide a dónde ir basándose en el estado actual.

```
edge.py

def decidir_camino(state: RAGAgentState) -> str:
    if state["needs_retrieval"]:
        return "retrieve"    # → va al nodo retrieve
    return "generate"        # → va al nodo generate

builder.add_conditional_edges(
    "analyze",               # Desde este nodo
    decidir_camino,          # Esta función decide
    {
        "retrieve": "retrieve",
        "generate": "generate",
    }
)
```

StateGraph

Es el constructor del grafo. Le dices qué estado usa, agregas nodos y aristas, y llamas a `.compile()` para obtener el objeto ejecutable.

```
state_graph.py

# Construir
builder = StateGraph(RAGAgentState)
builder.add_node("analyze", node_analyze)
builder.add_node("retrieve", node_retrieve)
builder.add_node("generate", node_generate)

builder.add_edge(START, "analyze")
builder.add_conditional_edges("analyze", decidir_camino, {...})
builder.add_edge("retrieve", "generate")
builder.add_edge("generate", END)

# Compilar: convierte el grafo en algo ejecutable
agent = builder.compile()

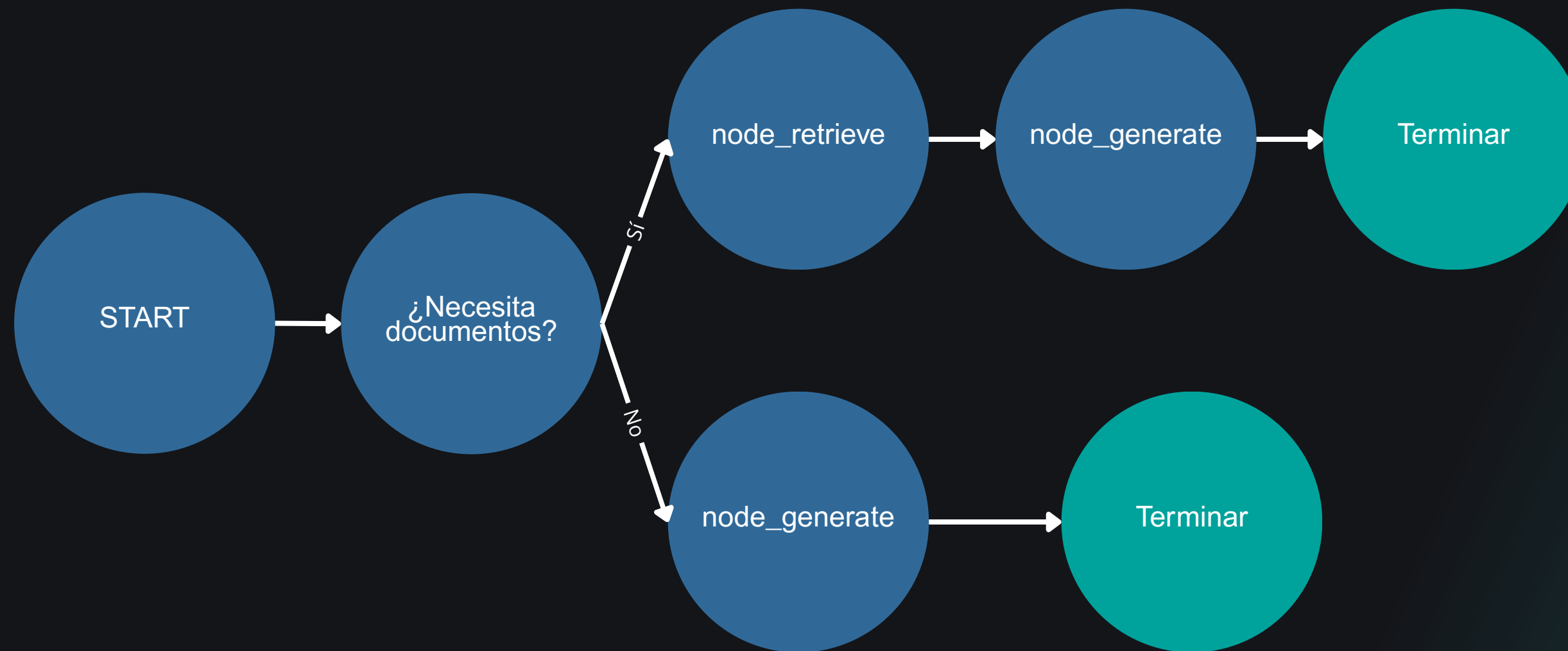
# Usar
result = agent.invoke(estado_inicial)
```



LCEL vs LangGraph

LCEL	LangGraph
Mismo flujo de principio a fin	El flujo cambia según el contexto
No necesita tomar decisiones	Necesita decidir entre múltiples caminos
No necesita repetir pasos	Se necesitan repetir pasos hasta que algo sea correcto
Tareas simples: prompt → LLM → Respuesta	El sistema se puede corregir el mismo

¿Qué usaremos en nuestro proyecto?





¿Qué no es LangGraph?

- No es un reemplazo de LangChain
- No es magia.
- No siempre es necesario.
- No solo es para agentes.



Conclusión

Es la herramienta que convierte llamadas aisladas a un LLM en sistemas de IA que toman decisiones, se repiten y se corrigen como lo haría un humano.



Gracias



www.devtalles.com



[/ricardocuellar](https://www.linkedin.com/company/ricardocuellar)



[@DevTalles](https://twitter.com/DevTalles)